FastAPI - The Complete Guide: From Zero to Production

A Comprehensive Reference for Learning, Interviews, and Deployment

---

Table of Contents

30. Coding Challenges for Interviews

---

1. What is FastAPI?

FastAPI is a modern, high-performance Python web framework designed for building APIs. It was created by **Sebastián Ramírez** and released in **2018** as an open-source framework.

In simple words, think of FastAPI as a tool that helps you create web APIs (the bridge between your app and the server) using Python. It's like a super-powered assistant that writes most of the boring code for you, catches your mistakes before they become bugs, and makes everything run really fast.

Key Facts:

- Built on top of **Starlette** (for web handling and async support) and **Pydantic** (for data validation)
- Requires **Python 3.8+**
- Fully compatible with **OpenAPI** (formerly Swagger) and **JSON Schema** standards
- Used by major companies like **Microsoft, Uber, Netflix, and many MNCs**
- FastAPI adoption increased by **141% in 2024**, making it one of the fastest-growing Python frameworks

What Does "Fast" Mean Here?

- **Fast Performance**: On par with Node.js and Go — one of the fastest Python frameworks available
- **Fast to Code**: Increases development speed by 200-300% compared to traditional frameworks
- **Fewer Bugs**: Reduces developer-caused errors by about 40%

---

2. Why FastAPI? Key Features

2.1 Automatic Documentation

When you write your API, FastAPI automatically creates two beautiful documentation pages:

- **Swagger UI** at `/docs` — Interactive, lets you test API endpoints right in the browser
- **ReDoc** at `/redoc` — Clean, readable documentation

You don't have to write a single line of documentation code!

2.2 Data Validation with Pydantic

FastAPI uses Python type hints along with Pydantic to automatically check if the data coming into your API is correct. If someone sends wrong data, FastAPI returns a clear error message.

2.3 Async Support (Non-Blocking)

FastAPI supports Python's `async` and `await` keywords, which means your API can handle thousands of requests at the same time without getting stuck.

2.4 Dependency Injection

A built-in system that lets you share common logic (like database connections, authentication checks) across your API endpoints without repeating code.

2.5 Security Built-in

Comes with tools for OAuth2, JWT tokens, API keys, and other security schemes out of the box.

2.6 Standards-Based

Fully compatible with OpenAPI and JSON Schema — industry standards for API design.

2.7 Editor Support

Amazing autocomplete and type checking in editors like VS Code, PyCharm, etc., because it uses Python type hints.

---

3. Installation and Setup

3.1 Basic Installation

bash

```bash
# Create a virtual environment (recommended)
python -m venv venv

# Activate virtual environment
# On Windows:
venv\Scripts\activate
# On Linux/Mac:
source venv/bin/activate

# Install FastAPI with all standard dependencies
pip install "fastapi[standard]"
```

What does `[standard]` include?

- `uvicorn` — The ASGI server to run your app
- `python-multipart` — For form data handling

- `email-validator` — For email validation
- Other commonly needed packages

3.2 Minimal Installation

bash

```bash
# Just the core framework (you'll need to install uvicorn separately)
pip install fastapi
pip install uvicorn
```

3.3 Your First FastAPI Application

Create a file called `main.py`:

python

```python
from fastapi import FastAPI

# Create a FastAPI application instance
app = FastAPI()

# Define a route for the root URL
@app.get("/")
def read_root():
    return {"message": "Hello, FastAPI!"}

# Define a route with a path parameter
@app.get("/items/{item_id}")
def read_item(item_id: int, q: str = None):
    return {"item_id": item_id, "query": q}
```

3.4 Running the Application

bash

```bash
# Development mode (auto-reload on code changes)
fastapi dev main.py

# OR using uvicorn directly
uvicorn main:app --reload --port 8000
```

Now open your browser:

- API: `http://127.0.0.1:8000`
- Swagger Docs: `http://127.0.0.1:8000/docs`
- ReDoc: `http://127.0.0.1:8000/redoc`

4. Core Concepts - Basic to Advanced

4.1 ASGI vs WSGI — Why It Matters

WSGI (Web Server Gateway Interface):

- Used by Flask, Django (traditional)
- Handles one request at a time per worker (synchronous)
- Simple but limited for real-time applications

ASGI (Asynchronous Server Gateway Interface):

- Used by FastAPI (via Starlette)
- Can handle multiple requests at the same time (asynchronous)
- Perfect for real-time apps, WebSockets, long-running tasks
- Better performance for I/O-heavy operations (database calls, API calls)

4.2 Understanding Uvicorn

Uvicorn is the ASGI server that runs your FastAPI application. Think of it as the engine that powers your API.

```bash
# Basic usage
uvicorn main:app

# With options
uvicorn main:app --host 0.0.0.0 --port 8000 --reload --workers 4
```

- `main:app` — The `app` object inside `main.py`
- `--host 0.0.0.0` — Accept connections from any IP
- `--port 8000` — Run on port 8000
- `--reload` — Auto-restart on code changes (development only!)
- `--workers 4` — Run 4 worker processes (production)

4.3 Type Hints — The Foundation

FastAPI heavily uses Python type hints. If you're not familiar with them, here's a quick primer:

```python
```

```python
# Basic type hints
def greet(name: str) -> str:
    return f"Hello, {name}"

# Optional types
from typing import Optional
def search(query: str, limit: Optional[int] = None):
    pass

# List and Dict types
from typing import List, Dict
def get_items() -> List[Dict[str, str]]:
    return [{"name": "Item 1"}]

# Union types (Python 3.10+)
def process(value: int | str):
    pass
```

---

5. Routing and HTTP Methods

5.1 Understanding HTTP Methods

Method	Purpose	Example
GET	Read/retrieve data	Get a list of users
POST	Create new data	Create a new user
PUT	Update entire resource	Replace user data completely
PATCH	Partially update resource	Update only user's email
DELETE	Remove data	Delete a user

5.2 Defining Routes

```python
```

```python
from fastapi import FastAPI

app = FastAPI()

# GET request
@app.get("/users")
def get_users():
    return [{"name": "Alice"}, {"name": "Bob"}]

# POST request
@app.post("/users")
def create_user(name: str):
    return {"name": name, "status": "created"}

# PUT request
@app.put("/users/{user_id}")
def update_user(user_id: int, name: str):
    return {"user_id": user_id, "name": name, "status": "updated"}

# PATCH request
@app.patch("/users/{user_id}")
def partial_update(user_id: int, name: str = None):
    return {"user_id": user_id, "updated_fields": {"name": name}}

# DELETE request
@app.delete("/users/{user_id}")
def delete_user(user_id: int):
    return {"user_id": user_id, "status": "deleted"}
```

5.3 Route with Tags and Description

python

```python
@app.get(
    "/users",
    tags=["Users"],
    summary="Get all users",
    description="Returns a list of all registered users in the system",
    response_description="List of users"
)
def get_users():
    return [{"name": "Alice"}]
```

5.4 APIRouter — Organizing Routes

When your application grows, you can organize routes into separate files using `APIRouter`:

python

```python
# routers/users.py
from fastapi import APIRouter

router = APIRouter(prefix="/users", tags=["Users"])

@router.get("/")
def get_users():
    return [{"name": "Alice"}]

@router.get("/{user_id}")
def get_user(user_id: int):
    return {"user_id": user_id, "name": "Alice"}
```

python

```python
# main.py
from fastapi import FastAPI
from routers import users

app = FastAPI()
app.include_router(users.router)
```

---

6. Request Handling

6.1 Path Parameters

Path parameters are parts of the URL that are variable:

python

```python
# Basic path parameter
@app.get("/users/{user_id}")
def get_user(user_id: int):
    return {"user_id": user_id}

# Multiple path parameters
@app.get("/users/{user_id}/posts/{post_id}")
def get_user_post(user_id: int, post_id: int):
    return {"user_id": user_id, "post_id": post_id}

# Path parameter with validation
from fastapi import Path

@app.get("/items/{item_id}")
def get_item(
    item_id: int = Path(
        title="The ID of the item",
        description="Must be a positive integer",
        gt=0,          # greater than 0
        le=1000        # less than or equal to 1000
    )
):
    return {"item_id": item_id}
```

6.2 Query Parameters

Query parameters are the key-value pairs after `?` in the URL:

python

```python
# URL: /items?skip=0&limit=10
@app.get("/items")
def get_items(skip: int = 0, limit: int = 10):
    return {"skip": skip, "limit": limit}

# Required query parameter (no default value)
@app.get("/search")
def search(query: str):
    return {"results": f"Searching for: {query}"}

# Optional query parameter
from typing import Optional

@app.get("/items")
def get_items(
    category: Optional[str] = None,
    min_price: float = 0,
    max_price: float = 99999
):
    return {
        "category": category,
        "price_range": [min_price, max_price]
    }

# Query parameter with validation
from fastapi import Query

@app.get("/items")
def get_items(
    q: str = Query(
        default=None,
        min_length=3,
        max_length=50,
        regex="^[a-zA-Z]+$",
        description="Search query string"
    )
):
    return {"query": q}
```

6.3 Request Body

For POST, PUT, PATCH requests, data is sent in the request body:

python

```python
from pydantic import BaseModel

class Item(BaseModel):
    name: str
    price: float
    description: str = None
    in_stock: bool = True

@app.post("/items")
def create_item(item: Item):
    return {"item": item, "status": "created"}

# Multiple body parameters
class User(BaseModel):
    name: str
    email: str

@app.post("/orders")
def create_order(user: User, item: Item, quantity: int):
    return {
        "user": user,
        "item": item,
        "quantity": quantity
    }
```

6.4 Form Data

python

```python
from fastapi import Form

@app.post("/login")
def login(username: str = Form(...), password: str = Form(...)):
    return {"username": username}
```

6.5 Headers and Cookies

python

```python
from fastapi import Header, Cookie

@app.get("/items")
def get_items(
    user_agent: str = Header(default=None),
    accept_language: str = Header(default=None),
    session_id: str = Cookie(default=None)
):
    return {
        "user_agent": user_agent,
        "language": accept_language,
        "session": session_id
    }
```

---

7. Pydantic Models and Data Validation

7.1 Basic Pydantic Model

Pydantic is FastAPI's backbone for data validation. It automatically checks if incoming data matches the expected format.

python

```python
from pydantic import BaseModel, Field
from typing import Optional, List
from datetime import datetime

class User(BaseModel):
    name: str
    email: str
    age: int
    is_active: bool = True
    created_at: datetime = None

# Usage
user_data = {"name": "Alice", "email": "alice@example.com", "age": 25}
user = User(**user_data)
print(user.name)  # Alice
```

7.2 Field Validation

python

```python
from pydantic import BaseModel, Field, validator

class Product(BaseModel):
    name: str = Field(
        min_length=1,
        max_length=100,
        description="Product name"
    )
    price: float = Field(
        gt=0,              # greater than 0
        description="Price must be positive"
    )
    quantity: int = Field(
        ge=0,              # greater than or equal to 0
        le=10000,
        default=0
    )
    tags: List[str] = Field(default=[], max_length=5)
```

7.3 Custom Validators

python

```python
from pydantic import BaseModel, validator, root_validator

class User(BaseModel):
    username: str
    password: str
    confirm_password: str
    email: str

    # Field-level validator
    @validator("password")
    def password_strength(cls, value):
        if len(value) < 8:
            raise ValueError("Password must be at least 8 characters")
        if not any(c.isupper() for c in value):
            raise ValueError("Password must have at least one uppercase letter")
        return value

    # Validator that depends on another field
    @validator("confirm_password")
    def passwords_match(cls, v, values):
        if "password" in values and v != values["password"]:
            raise ValueError("Passwords don't match")
        return v

    # Email validator
    @validator("email")
    def valid_email(cls, v):
        if "@" not in v:
            raise ValueError("Invalid email address")
        return v.lower()

    # Root validator (access all fields)
    @root_validator
    def check_username_not_in_password(cls, values):
        username = values.get("username", "")
        password = values.get("password", "")
        if username.lower() in password.lower():
            raise ValueError("Password should not contain username")
        return values
```

7.4 Nested Models

```python
```

```python
class Address(BaseModel):
    street: str
    city: str
    state: str
    zip_code: str

class Company(BaseModel):
    name: str
    address: Address

class Employee(BaseModel):
    name: str
    email: str
    company: Company
    skills: List[str]

# This validates deeply nested data automatically!
```

7.5 Model Configuration with Config class

python

```python
class Item(BaseModel):
    name: str
    price: float

    class Config:
        # Allow creating model from ORM objects
        from_attributes = True  # (Pydantic v2) or orm_mode = True (v1)
        # Provide example for documentation
        json_schema_extra = {
            "example": {
                "name": "Laptop",
                "price": 999.99
            }
        }
```

---

8. Response Models and Status Codes

8.1 Response Model

The `response_model` ensures your API only returns the data you want:

python

```python
class UserIn(BaseModel):
    name: str
    email: str
    password: str  # We receive this

class UserOut(BaseModel):
    name: str
    email: str
    # No password field — it won't be returned!

@app.post("/users", response_model=UserOut, status_code=201)
def create_user(user: UserIn):
    # Even if we return the full user object, FastAPI will filter
    # and only send fields defined in UserOut
    return user
```

8.2 Status Codes

python

```python
from fastapi import status

@app.post("/items", status_code=status.HTTP_201_CREATED)
def create_item(item: Item):
    return item

@app.delete("/items/{item_id}", status_code=status.HTTP_204_NO_CONTENT)
def delete_item(item_id: int):
    return None
```

Common Status Codes:

Code	Meaning	When to Use
200	OK	Successful GET, PUT, PATCH
201	Created	Successful POST (resource created)
204	No Content	Successful DELETE
400	Bad Request	Invalid data sent by client
401	Unauthorized	Not authenticated
403	Forbidden	Authenticated but no permission
404	Not Found	Resource doesn't exist
422	Unprocessable Entity	Validation error
500	Internal Server Error	Server-side error

8.3 Custom Response Types

```python
from fastapi.responses import JSONResponse, HTMLResponse, RedirectResponse

@app.get("/custom")
def custom_response():
    return JSONResponse(
        content={"message": "Custom response"},
        status_code=200,
        headers={"X-Custom-Header": "my-value"}
    )

@app.get("/html", response_class=HTMLResponse)
def html_page():
    return "<h1>Hello from HTML!</h1>"

@app.get("/redirect")
def redirect():
    return RedirectResponse(url="/docs")
```

---

9. Dependency Injection

Dependency Injection (DI) is one of FastAPI's most powerful features. It's a design pattern where

your functions receive their required "dependencies" from outside rather than creating them internally. This makes your code reusable, testable, and clean.

9.1 Basic Dependency

python

```python
from fastapi import Depends

# This is a dependency function
def get_query_params(skip: int = 0, limit: int = 10):
    return {"skip": skip, "limit": limit}

# Inject the dependency
@app.get("/items")
def get_items(params: dict = Depends(get_query_params)):
    return {"params": params}

@app.get("/users")
def get_users(params: dict = Depends(get_query_params)):
    return {"params": params}
# Both endpoints now share the same logic!
```

9.2 Database Session Dependency

python

```python
from sqlalchemy.orm import Session

def get_db():
    db = SessionLocal()
    try:
        yield db  # "yield" means this is a generator dependency
    finally:
        db.close()  # This runs AFTER the request is done

@app.get("/users")
def get_users(db: Session = Depends(get_db)):
    users = db.query(User).all()
    return users
```

9.3 Authentication Dependency

python

```python
from fastapi import Depends, HTTPException
from fastapi.security import OAuth2PasswordBearer

oauth2_scheme = OAuth2PasswordBearer(tokenUrl="token")

def get_current_user(token: str = Depends(oauth2_scheme)):
    user = decode_token(token)
    if not user:
        raise HTTPException(status_code=401, detail="Invalid token")
    return user

@app.get("/profile")
def get_profile(current_user: User = Depends(get_current_user)):
    return current_user
```

9.4 Sub-Dependencies (Dependencies that depend on other dependencies)

python

```python
def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

def get_current_user(db: Session = Depends(get_db)):
    # This dependency itself depends on get_db
    user = db.query(User).first()
    return user

def get_admin_user(current_user: User = Depends(get_current_user)):
    # This depends on get_current_user, which depends on get_db
    if not current_user.is_admin:
        raise HTTPException(status_code=403, detail="Not an admin")
    return current_user

@app.get("/admin/dashboard")
def admin_dashboard(admin: User = Depends(get_admin_user)):
    return {"message": f"Welcome, {admin.name}"}
```

9.5 Class-Based Dependencies

python

```python
class Pagination:
    def __init__(self, skip: int = 0, limit: int = 10):
        self.skip = skip
        self.limit = limit

@app.get("/items")
def get_items(pagination: Pagination = Depends()):
    return {
        "skip": pagination.skip,
        "limit": pagination.limit
    }
```

9.6 Global Dependencies

python

```python
# Apply dependency to ALL routes
app = FastAPI(dependencies=[Depends(verify_api_key)])

# Apply dependency to all routes in a router
router = APIRouter(dependencies=[Depends(get_current_user)])
```

---

10. Database Integration

10.1 SQLAlchemy Setup

python

```python
# database.py
from sqlalchemy import create_engine
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import sessionmaker

DATABASE_URL = "sqlite:///./app.db"
# For PostgreSQL: "postgresql://user:password@localhost:5432/dbname"

engine = create_engine(DATABASE_URL, connect_args={"check_same_thread": False})
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
Base = declarative_base()
```

python

```python
# models.py
from sqlalchemy import Column, Integer, String, Boolean, ForeignKey
from sqlalchemy.orm import relationship
from database import Base

class User(Base):
    __tablename__ = "users"

    id = Column(Integer, primary_key=True, index=True)
    name = Column(String, index=True)
    email = Column(String, unique=True, index=True)
    is_active = Column(Boolean, default=True)

    # Relationship
    posts = relationship("Post", back_populates="author")

class Post(Base):
    __tablename__ = "posts"

    id = Column(Integer, primary_key=True, index=True)
    title = Column(String)
    content = Column(String)
    author_id = Column(Integer, ForeignKey("users.id"))

    author = relationship("User", back_populates="posts")
```

python

```python
# schemas.py (Pydantic models)
from pydantic import BaseModel
from typing import List, Optional

class PostBase(BaseModel):
    title: str
    content: str

class PostCreate(PostBase):
    pass

class Post(PostBase):
    id: int
    author_id: int

    class Config:
        from_attributes = True

class UserCreate(BaseModel):
    name: str
    email: str

class User(BaseModel):
    id: int
    name: str
    email: str
    is_active: bool
    posts: List[Post] = []

    class Config:
        from_attributes = True
```

python

```python
# crud.py (Database operations)
from sqlalchemy.orm import Session
import models, schemas

def get_user(db: Session, user_id: int):
    return db.query(models.User).filter(models.User.id == user_id).first()

def get_users(db: Session, skip: int = 0, limit: int = 100):
    return db.query(models.User).offset(skip).limit(limit).all()

def create_user(db: Session, user: schemas.UserCreate):
    db_user = models.User(name=user.name, email=user.email)
    db.add(db_user)
    db.commit()
    db.refresh(db_user)
    return db_user
```

python

```python
# main.py
from fastapi import FastAPI, Depends, HTTPException
from sqlalchemy.orm import Session
import models, schemas, crud
from database import SessionLocal, engine

models.Base.metadata.create_all(bind=engine)

app = FastAPI()

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

@app.post("/users/", response_model=schemas.User)
def create_user(user: schemas.UserCreate, db: Session = Depends(get_db)):
    db_user = crud.get_user_by_email(db, email=user.email)
    if db_user:
        raise HTTPException(status_code=400, detail="Email already registered")
    return crud.create_user(db=db, user=user)

@app.get("/users/", response_model=list[schemas.User])
def read_users(skip: int = 0, limit: int = 100, db: Session = Depends(get_db)):
    return crud.get_users(db, skip=skip, limit=limit)
```

10.2 SQLModel (Recommended — Combines SQLAlchemy + Pydantic)

python

```python
from sqlmodel import SQLModel, Field, Session, create_engine, select
from typing import Optional

class User(SQLModel, table=True):
    id: Optional[int] = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    email: str = Field(unique=True)
    age: int

engine = create_engine("sqlite:///database.db")
SQLModel.metadata.create_all(engine)

def get_session():
    with Session(engine) as session:
        yield session

@app.post("/users")
def create_user(user: User, session: Session = Depends(get_session)):
    session.add(user)
    session.commit()
    session.refresh(user)
    return user
```

10.3 Async Database with asyncpg (PostgreSQL)

```python
from sqlalchemy.ext.asyncio import create_async_engine, AsyncSession
from sqlalchemy.orm import sessionmaker

DATABASE_URL = "postgresql+asyncpg://user:password@localhost:5432/dbname"

engine = create_async_engine(DATABASE_URL, echo=True)
async_session = sessionmaker(engine, class_=AsyncSession, expire_on_commit=False)

async def get_db():
    async with async_session() as session:
        yield session

@app.get("/users")
async def get_users(db: AsyncSession = Depends(get_db)):
    result = await db.execute(select(User))
    return result.scalars().all()
```

10.4 Database Migrations with Alembic

bash

```bash
# Install Alembic
pip install alembic

# Initialize Alembic
alembic init alembic

# Auto-generate migration
alembic revision --autogenerate -m "Create users table"

# Apply migration
alembic upgrade head

# Rollback
alembic downgrade -1
```

---

11. Authentication and Security

11.1 OAuth2 with JWT (JSON Web Tokens) — Full Implementation

python

```python
from datetime import datetime, timedelta
from typing import Optional
from fastapi import FastAPI, Depends, HTTPException, status
from fastapi.security import OAuth2PasswordBearer, OAuth2PasswordRequestForm
from jose import JWTError, jwt
from passlib.context import CryptContext
from pydantic import BaseModel

# Configuration
SECRET_KEY = "your-secret-key-change-in-production"
ALGORITHM = "HS256"
ACCESS_TOKEN_EXPIRE_MINUTES = 30

# Password hashing
pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")

# OAuth2 scheme
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="token")

app = FastAPI()

# Models
class Token(BaseModel):
    access_token: str
    token_type: str

class TokenData(BaseModel):
    username: Optional[str] = None

class User(BaseModel):
    username: str
    email: str
    disabled: Optional[bool] = None

class UserInDB(User):
    hashed_password: str

# Fake database
fake_users_db = {
    "johndoe": {
        "username": "johndoe",
        "email": "john@example.com",
        "hashed_password": pwd_context.hash("secret123"),
        "disabled": False,
    }
}
```

```python
# Helper functions
def verify_password(plain_password, hashed_password):
    return pwd_context.verify(plain_password, hashed_password)

def get_user(db, username: str):
    if username in db:
        return UserInDB(**db[username])

def authenticate_user(db, username: str, password: str):
    user = get_user(db, username)
    if not user or not verify_password(password, user.hashed_password):
        return False
    return user

def create_access_token(data: dict, expires_delta: Optional[timedelta] = None):
    to_encode = data.copy()
    expire = datetime.utcnow() + (expires_delta or timedelta(minutes=15))
    to_encode.update({"exp": expire})
    return jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)

async def get_current_user(token: str = Depends(oauth2_scheme)):
    credentials_exception = HTTPException(
        status_code=status.HTTP_401_UNAUTHORIZED,
        detail="Could not validate credentials",
        headers={"WWW-Authenticate": "Bearer"},
    )
    try:
        payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
        username: str = payload.get("sub")
        if username is None:
            raise credentials_exception
    except JWTError:
        raise credentials_exception
    user = get_user(fake_users_db, username=username)
    if user is None:
        raise credentials_exception
    return user

# Login endpoint
@app.post("/token", response_model=Token)
async def login(form_data: OAuth2PasswordRequestForm = Depends()):
    user = authenticate_user(fake_users_db, form_data.username, form_data.password)
    if not user:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Incorrect username or password",
```

```
        )
    access_token = create_access_token(
        data={"sub": user.username},
        expires_delta=timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)
    )
    return {"access_token": access_token, "token_type": "bearer"}

# Protected endpoint
@app.get("/users/me", response_model=User)
async def read_users_me(current_user: User = Depends(get_current_user)):
    return current_user
```

11.2 API Key Authentication

python

```
from fastapi.security import APIKeyHeader

api_key_header = APIKeyHeader(name="X-API-Key")

async def verify_api_key(api_key: str = Depends(api_key_header)):
    if api_key != "my-secret-api-key":
        raise HTTPException(status_code=403, detail="Invalid API Key")
    return api_key

@app.get("/secure-data", dependencies=[Depends(verify_api_key)])
def get_secure_data():
    return {"data": "This is secure!"}
```

11.3 Role-Based Access Control (RBAC)

python

```python
from enum import Enum

class Role(str, Enum):
    ADMIN = "admin"
    USER = "user"
    MODERATOR = "moderator"

def require_role(required_role: Role):
    def role_checker(current_user: User = Depends(get_current_user)):
        if current_user.role != required_role:
            raise HTTPException(
                status_code=403,
                detail=f"Role '{required_role}' required"
            )
        return current_user
    return role_checker

@app.get("/admin/settings")
def admin_settings(user: User = Depends(require_role(Role.ADMIN))):
    return {"settings": "admin only data"}
```

---

12. Middleware

Middleware is code that runs before every request and after every response. Think of it as a gatekeeper or a filter that processes everything coming in and going out.

12.1 Custom Middleware

python

```python
import time
from fastapi import Request

@app.middleware("http")
async def add_process_time_header(request: Request, call_next):
    start_time = time.time()
    response = await call_next(request)
    process_time = time.time() - start_time
    response.headers["X-Process-Time"] = str(process_time)
    return response
```

12.2 Logging Middleware

python

```python
import logging

logger = logging.getLogger("api")

@app.middleware("http")
async def log_requests(request: Request, call_next):
    logger.info(f"Request: {request.method} {request.url}")
    response = await call_next(request)
    logger.info(f"Response: {response.status_code}")
    return response
```

12.3 Rate Limiting Middleware (using slowapi)

python

```python
from slowapi import Limiter
from slowapi.util import get_remote_address

limiter = Limiter(key_func=get_remote_address)
app.state.limiter = limiter

@app.get("/items")
@limiter.limit("5/minute")
async def get_items(request: Request):
    return {"message": "Rate limited endpoint"}
```

12.4 CORS Middleware

python

```python
from fastapi.middleware.cors import CORSMiddleware

app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:3000"],  # Frontend URL
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

---

13. Background Tasks

Background tasks let you run functions AFTER sending the response back to the client. This is useful for operations like sending emails, processing data, or writing logs that the user doesn't

need to wait for.

13.1 Basic Background Task

```python
from fastapi import BackgroundTasks

def send_email(email: str, message: str):
    # Simulate sending email (this runs in background)
    import time
    time.sleep(5)
    print(f"Email sent to {email}: {message}")

@app.post("/send-notification/{email}")
async def send_notification(email: str, background_tasks: BackgroundTasks):
    background_tasks.add_task(send_email, email, message="Welcome!")
    return {"message": "Notification will be sent in background"}
    # Response is returned immediately, email sends in background
```

13.2 Background Tasks in Dependencies

```python
def log_action(background_tasks: BackgroundTasks, action: str):
    def write_log(action: str):
        with open("log.txt", "a") as f:
            f.write(f"{action}\n")
    background_tasks.add_task(write_log, action)

@app.post("/items")
async def create_item(item: Item, background_tasks: BackgroundTasks):
    log_action(background_tasks, f"Created item: {item.name}")
    return item
```

13.3 For Heavy Tasks — Use Celery

For CPU-intensive or long-running tasks, use Celery instead:

```python
```

```python
from celery import Celery

celery_app = Celery("tasks", broker="redis://localhost:6379")

@celery_app.task
def process_heavy_data(data):
    # This runs on a separate Celery worker
    import time
    time.sleep(60)
    return "Done processing"

@app.post("/process")
async def process(data: dict):
    task = process_heavy_data.delay(data)
    return {"task_id": task.id, "status": "processing"}
```

---

14. WebSockets

WebSockets enable real-time, two-way communication between client and server. Unlike regular HTTP where the client must always ask first, WebSockets allow the server to push data to clients at any time.

14.1 Basic WebSocket

python

```python
from fastapi import WebSocket, WebSocketDisconnect

@app.websocket("/ws")
async def websocket_endpoint(websocket: WebSocket):
    await websocket.accept()
    try:
        while True:
            data = await websocket.receive_text()
            await websocket.send_text(f"Echo: {data}")
    except WebSocketDisconnect:
        print("Client disconnected")
```

14.2 WebSocket with Connection Manager (Chat Room)

python

```python
from typing import List

class ConnectionManager:
    def __init__(self):
        self.active_connections: List[WebSocket] = []

    async def connect(self, websocket: WebSocket):
        await websocket.accept()
        self.active_connections.append(websocket)

    def disconnect(self, websocket: WebSocket):
        self.active_connections.remove(websocket)

    async def broadcast(self, message: str):
        for connection in self.active_connections:
            await connection.send_text(message)

manager = ConnectionManager()

@app.websocket("/ws/{client_id}")
async def websocket_endpoint(websocket: WebSocket, client_id: int):
    await manager.connect(websocket)
    try:
        while True:
            data = await websocket.receive_text()
            await manager.broadcast(f"Client #{client_id}: {data}")
    except WebSocketDisconnect:
        manager.disconnect(websocket)
        await manager.broadcast(f"Client #{client_id} left the chat")
```

---

15. File Handling

15.1 File Upload

```python
```

```python
from fastapi import File, UploadFile

# Single file upload
@app.post("/upload")
async def upload_file(file: UploadFile = File(...)):
    contents = await file.read()
    with open(f"uploads/{file.filename}", "wb") as f:
        f.write(contents)
    return {
        "filename": file.filename,
        "size": len(contents),
        "content_type": file.content_type
    }

# Multiple file upload
@app.post("/upload-multiple")
async def upload_files(files: list[UploadFile] = File(...)):
    filenames = []
    for file in files:
        contents = await file.read()
        filenames.append(file.filename)
    return {"filenames": filenames}
```

15.2 File Download

python

```python
from fastapi.responses import FileResponse

@app.get("/download/{filename}")
def download_file(filename: str):
    file_path = f"files/{filename}"
    return FileResponse(
        path=file_path,
        filename=filename,
        media_type="application/octet-stream"
    )
```

---

16. Error Handling and Custom Exceptions

16.1 HTTPException

python

```python
from fastapi import HTTPException

@app.get("/items/{item_id}")
def get_item(item_id: int):
    if item_id not in items_db:
        raise HTTPException(
            status_code=404,
            detail="Item not found",
            headers={"X-Error": "Item not in database"}
        )
    return items_db[item_id]
```

16.2 Custom Exception Handlers

python

```python
from fastapi import Request
from fastapi.responses import JSONResponse

class ItemNotFoundException(Exception):
    def __init__(self, item_id: int):
        self.item_id = item_id

@app.exception_handler(ItemNotFoundException)
async def item_not_found_handler(request: Request, exc: ItemNotFoundException):
    return JSONResponse(
        status_code=404,
        content={
            "error": "item_not_found",
            "message": f"Item with ID {exc.item_id} was not found",
            "path": str(request.url)
        }
    )

@app.get("/items/{item_id}")
def get_item(item_id: int):
    if item_id not in items_db:
        raise ItemNotFoundException(item_id)
    return items_db[item_id]
```

16.3 Validation Error Handler

python

```python
from fastapi.exceptions import RequestValidationError
from fastapi.responses import JSONResponse

@app.exception_handler(RequestValidationError)
async def validation_exception_handler(request: Request, exc: RequestValidationError
    return JSONResponse(
        status_code=422,
        content={
            "error": "validation_error",
            "details": exc.errors(),
            "body": exc.body
        }
    )
```

---

17. CORS

CORS (Cross-Origin Resource Sharing) allows your frontend (running on localhost:3000) to talk to your backend API (running on localhost:8000).

```python
from fastapi.middleware.cors import CORSMiddleware

origins = [
    "http://localhost:3000",       # React dev server
    "http://localhost:8080",       # Vue dev server
    "https://mywebsite.com",      # Production frontend
]

app.add_middleware(
    CORSMiddleware,
    allow_origins=origins,           # Which origins can access
    allow_credentials=True,          # Allow cookies
    allow_methods=["*"],             # Allow all HTTP methods
    allow_headers=["*"],             # Allow all headers
)
```

---

18. Async Programming in FastAPI

18.1 When to Use async def vs def

```python
```

```python
# USE async def when:
# - Making database calls with async drivers
# - Calling external APIs
# - File I/O operations
# - Any operation that involves waiting

@app.get("/async-data")
async def get_async_data():
    data = await fetch_from_database()    # Non-blocking
    result = await call_external_api()     # Non-blocking
    return {"data": data, "result": result}

# USE regular def when:
# - Simple calculations
# - CPU-bound operations
# - Using synchronous libraries (requests, psycopg2)

@app.get("/sync-data")
def get_sync_data():
    result = heavy_calculation()  # FastAPI runs this in a thread pool
    return {"result": result}
```

18.2 Understanding the Event Loop

python

```python
import asyncio
import httpx

# BAD: This blocks the event loop!
@app.get("/bad")
async def bad_endpoint():
    import requests  # Synchronous library
    response = requests.get("https://api.example.com")  # BLOCKS!
    return response.json()

# GOOD: This is non-blocking
@app.get("/good")
async def good_endpoint():
    async with httpx.AsyncClient() as client:
        response = await client.get("https://api.example.com")  # Non-blocking
    return response.json()

# ALSO GOOD: Use regular def for blocking operations
@app.get("/also-good")
def also_good_endpoint():
    import requests
    response = requests.get("https://api.example.com")
    return response.json()
    # FastAPI automatically runs this in a thread pool
```

18.3 Concurrent Requests with asyncio.gather

python

```python
import httpx

@app.get("/dashboard")
async def get_dashboard():
    async with httpx.AsyncClient() as client:
        # Run all three API calls at the SAME TIME
        users, orders, stats = await asyncio.gather(
            client.get("https://api.example.com/users"),
            client.get("https://api.example.com/orders"),
            client.get("https://api.example.com/stats"),
        )
    return {
        "users": users.json(),
        "orders": orders.json(),
        "stats": stats.json()
    }
```

19. Testing FastAPI Applications

19.1 Basic Testing with TestClient

python

```python
# test_main.py
from fastapi.testclient import TestClient
from main import app

client = TestClient(app)

def test_read_root():
    response = client.get("/")
    assert response.status_code == 200
    assert response.json() == {"message": "Hello, FastAPI!"}

def test_read_item():
    response = client.get("/items/1?q=test")
    assert response.status_code == 200
    data = response.json()
    assert data["item_id"] == 1
    assert data["query"] == "test"

def test_create_item():
    response = client.post(
        "/items",
        json={"name": "Laptop", "price": 999.99}
    )
    assert response.status_code == 201

def test_invalid_item():
    response = client.post(
        "/items",
        json={"name": "", "price": -10}  # Invalid data
    )
    assert response.status_code == 422  # Validation error
```

19.2 Testing with Dependency Overrides

python

```python
def override_get_db():
    db = TestSessionLocal()
    try:
        yield db
    finally:
        db.close()

app.dependency_overrides[get_db] = override_get_db

def test_create_user():
    response = client.post(
        "/users",
        json={"name": "Test User", "email": "test@example.com"}
    )
    assert response.status_code == 200
```

19.3 Async Testing

python

```python
import pytest
from httpx import AsyncClient

@pytest.mark.asyncio
async def test_async_endpoint():
    async with AsyncClient(app=app, base_url="http://test") as client:
        response = await client.get("/async-data")
        assert response.status_code == 200
```

19.4 Running Tests

bash

```
# Install pytest
pip install pytest httpx pytest-asyncio

# Run all tests
pytest

# Run with verbose output
pytest -v

# Run specific test file
pytest test_main.py

# Run with coverage
pip install pytest-cov
pytest --cov=app --cov-report=html
```

---

20. Project Structure and Best Practices

20.1 Recommended Project Structure

```
my_fastapi_project/
├── app/
│   ├── __init__.py
│   ├── main.py              # FastAPI app creation and startup
│   ├── config.py             # Configuration settings
│   ├── database.py           # Database connection setup
│   ├── models/               # SQLAlchemy models
│   │   ├── __init__.py
│   │   ├── user.py
│   │   └── post.py
│   ├── schemas/              # Pydantic schemas
│   │   ├── __init__.py
│   │   ├── user.py
│   │   └── post.py
│   ├── routers/              # API route handlers
│   │   ├── __init__.py
│   │   ├── users.py
│   │   └── posts.py
│   ├── services/             # Business logic
│   │   ├── __init__.py
│   │   ├── user_service.py
│   │   └── auth_service.py
│   ├── middleware/            # Custom middleware
│   │   └── logging.py
```

```
|   └── utils/                # Helper utilities
|       ├── security.py
|       └── email.py
├── tests/
|   ├── __init__.py
|   ├── conftest.py           # Test fixtures
|   ├── test_users.py
|   └── test_posts.py
├── alembic/                  # Database migrations
├── requirements.txt
├── Dockerfile
├── docker-compose.yml
├── .env
└── README.md
```

20.2 Configuration with Environment Variables

python

```python
# config.py
from pydantic_settings import BaseSettings
from functools import lru_cache

class Settings(BaseSettings):
    app_name: str = "My FastAPI App"
    debug: bool = False
    database_url: str
    secret_key: str
    algorithm: str = "HS256"
    access_token_expire_minutes: int = 30

    class Config:
        env_file = ".env"

@lru_cache()  # Cache settings so .env is read only once
def get_settings():
    return Settings()
```

env

```
# .env file
DATABASE_URL=postgresql://user:pass@localhost:5432/mydb
SECRET_KEY=your-secret-key-here
DEBUG=true
```

20.3 Lifespan Events (Startup and Shutdown)

```python
from contextlib import asynccontextmanager

@asynccontextmanager
async def lifespan(app: FastAPI):
    # Startup: runs when app starts
    print("Starting up...")
    # Initialize database, connect to services, etc.
    yield
    # Shutdown: runs when app stops
    print("Shutting down...")
    # Close connections, cleanup resources

app = FastAPI(lifespan=lifespan)
```

---

21. FastAPI vs Flask vs Django

Feature	FastAPI	Flask	Django
Type	Modern API framework	Micro framework	Full-stack framework
Speed	Very fast (async)	Moderate	Moderate
Async Support	Native (built-in)	Limited (via extensions)	Partial (Django 3.1+)
Data Validation	Automatic (Pydantic)	Manual	Django Forms/DRF Serializers
API Documentation	Auto-generated (Swagger/ReDoc)	Manual (Flask-Swagger)	DRF Browsable API
ORM	None built-in (use SQLAlchemy/SQLModel)	None built-in	Built-in Django ORM
Admin Panel	None built-in	None built-in	Built-in
Learning Curve	Easy-Medium	Very Easy	Medium-Hard
Best For	APIs, Microservices, ML models	Small apps, Prototypes	Full web apps, CMS, E-commerce
Community	Growing fast	Large, mature	Very large, mature

Feature	FastAPI	Flask	Django
Type Hints	Required (core feature)	Optional	Optional
WebSocket	Built-in	Via extensions	Channels

When to Choose FastAPI:

- Building REST APIs or microservices
- High-performance applications
- ML/AI model serving
- Real-time applications with WebSockets
- When you want automatic documentation

When to Choose Flask:

- Simple, small applications
- Quick prototypes
- When you want full control over everything

When to Choose Django:

- Full web applications with templates
- E-commerce platforms
- Content management systems
- When you need admin panel, ORM, and everything built-in

---

22. Deployment on AWS

22.1 Option 1: AWS EC2 (Virtual Machine)

This is the most straightforward approach — like setting up your own computer in the cloud.

Step-by-Step:

```bash
```

```
# 1. Launch an EC2 instance (Ubuntu 22.04)
# 2. SSH into your instance
ssh -i your-key.pem ubuntu@your-ec2-ip

# 3. Update and install dependencies
sudo apt update && sudo apt upgrade -y
sudo apt install -y python3 python3-pip python3-venv nginx

# 4. Create project directory
mkdir ~/fastapi-app && cd ~/fastapi-app

# 5. Create virtual environment
python3 -m venv venv
source venv/bin/activate

# 6. Upload your code (or clone from git)
git clone https://github.com/your-repo/your-app.git .

# 7. Install dependencies
pip install -r requirements.txt

# 8. Install production server
pip install gunicorn uvicorn

# 9. Test locally
uvicorn main:app --host 0.0.0.0 --port 8000
```

Nginx Configuration (Reverse Proxy):

nginx

```
# /etc/nginx/sites-available/fastapi
server {
    listen 80;
    server_name your-domain.com;

    location / {
        proxy_pass http://127.0.0.1:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}
```

Run with Gunicorn (Production):

bash

```
gunicorn main:app -k uvicorn.workers.UvicornWorker \
    --bind 0.0.0.0:8000 \
    --workers 4 \
    --access-logfile -
```

Create Systemd Service (Auto-start):

ini

```
# /etc/systemd/system/fastapi.service
[Unit]
Description=FastAPI Application
After=network.target

[Service]
User=ubuntu
WorkingDirectory=/home/ubuntu/fastapi-app
Environment="PATH=/home/ubuntu/fastapi-app/venv/bin"
ExecStart=/home/ubuntu/fastapi-app/venv/bin/gunicorn main:app \
    -k uvicorn.workers.UvicornWorker --bind 0.0.0.0:8000 --workers 4

[Install]
WantedBy=multi-user.target
```

bash

```
sudo systemctl enable fastapi
sudo systemctl start fastapi
```

22.2 Option 2: AWS Lambda (Serverless)

Serverless means you don't manage any servers. AWS runs your code only when needed and you pay only for what you use.

Key Library: Mangum — Bridges FastAPI (ASGI) with AWS Lambda

python

```python
# main.py
from fastapi import FastAPI
from mangum import Mangum

app = FastAPI()

@app.get("/")
def read_root():
    return {"message": "Hello from Lambda!"}

@app.get("/items/{item_id}")
def read_item(item_id: int):
    return {"item_id": item_id}

# AWS Lambda handler
handler = Mangum(app)
```

Dockerfile for Lambda:

dockerfile

```
FROM public.ecr.aws/lambda/python:3.12

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

CMD ["main.handler"]
```

Deployment Steps:

bash

```
# 1. Build Docker image
docker build -t fastapi-lambda .

# 2. Create ECR repository
aws ecr create-repository --repository-name fastapi-lambda

# 3. Tag and push image
aws ecr get-login-password --region us-east-1 | docker login --username AWS --passwo
docker tag fastapi-lambda:latest ACCOUNT_ID.dkr.ecr.us-east-1.amazonaws.com/fastapi-
docker push ACCOUNT_ID.dkr.ecr.us-east-1.amazonaws.com/fastapi-lambda:latest

# 4. Create Lambda function from ECR image
# 5. Add API Gateway trigger or Function URL
```

Lambda Limitations to Know:

- Maximum execution time: 15 minutes
- API Gateway timeout: 29 seconds
- Cold start delays (first request after idle is slower)
- Read-only file system (use /tmp for temporary writes)
- ZIP deployment size limit: 250 MB (use Docker/ECR for larger: up to 10 GB)

22.3 Option 3: AWS ECS Fargate (Container Service)

ECS Fargate is serverless container management. You give AWS a Docker container, and it runs it for you with auto-scaling.

Dockerfile:

dockerfile

```
FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 80

CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "80"]
```

Deployment Steps:

1. **Push Docker image to ECR** (same as Lambda steps above)
2. **Create ECS Cluster:**
   - Go to ECS Console → Create Cluster
   - Choose a name and VPC
3. **Create Task Definition:**
   - Set container name, image URI from ECR
   - Configure CPU (.25 vCPU) and Memory (0.5 GB)
   - Port mapping: 80
4. **Create Service:**
   - Select Fargate launch type
   - Configure networking (VPC, subnets, security groups)
   - Enable public IP
5. **Add Application Load Balancer (ALB):**
   - Create ALB with HTTP listener on port 80
   - Point target group to your ECS service
6. **Access your API:**
   - URL: `http://your-alb-dns.elb.amazonaws.com`
   - Docs: `http://your-alb-dns.elb.amazonaws.com/docs`

22.4 Option 4: AWS Elastic Beanstalk (PaaS)

Simplest AWS option — just upload your code and AWS handles everything.

```bash
```

```
# Install EB CLI
pip install awsebcli

# Initialize
eb init -p python-3.12 fastapi-app

# Create environment
eb create fastapi-env

# Deploy updates
eb deploy

# View logs
eb logs

# Open in browser
eb open

# Terminate when done
eb terminate fastapi-env
```

Comparison Table — AWS Deployment Options:

Feature	EC2	Lambda	ECS Fargate	Elastic Beanstalk
Control	Full	Limited	Good	Medium
Cost	Pay per hour (even idle)	Pay per request	Pay per task-second	Pay for underlying EC2
Scaling	Manual/Auto-scaling	Automatic	Automatic	Automatic
Complexity	High	Medium	Medium-High	Low
Cold Start	None	Yes	Minimal	None
Best For	Full control needed	Bursty/low traffic	Microservices	Quick deployment

---

23. Deployment on Azure

23.1 Option 1: Azure App Service (PaaS)

Azure App Service is the simplest way to deploy — similar to AWS Elastic Beanstalk.

Using Azure CLI:

```bash
# 1. Login to Azure
az login

# 2. Create a resource group
az group create --name myResourceGroup --location eastus

# 3. Create an App Service plan
az appservice plan create \
    --name myAppServicePlan \
    --resource-group myResourceGroup \
    --sku B1 \
    --is-linux

# 4. Create the web app
az webapp create \
    --resource-group myResourceGroup \
    --plan myAppServicePlan \
    --name my-fastapi-app \
    --runtime "PYTHON:3.12"

# 5. Configure startup command
az webapp config set \
    --resource-group myResourceGroup \
    --name my-fastapi-app \
    --startup-file "gunicorn -w 4 -k uvicorn.workers.UvicornWorker main:app"

# 6. Deploy your code
az webapp up --name my-fastapi-app --runtime "PYTHON:3.12"
```

Your app will be at: `https://my-fastapi-app.azurewebsites.net`

Using Docker on Azure App Service:

```dockerfile
FROM python:3.12
WORKDIR /code
COPY requirements.txt .
RUN pip3 install -r requirements.txt
COPY . .
EXPOSE 3100
ENTRYPOINT ["gunicorn", "-w", "4", "-k", "uvicorn.workers.UvicornWorker",
            "--bind", "0.0.0.0:3100", "main:app"]
```

bash

```bash
# Build and test locally
docker build -t fastapi-demo .
docker run --detach --publish 3100:3100 fastapi-demo

# Push to Azure Container Registry (ACR)
az acr create --resource-group myResourceGroup --name myacr --sku Basic
az acr build --registry myacr --image fastapi-demo .

# Deploy to App Service
az webapp create \
    --resource-group myResourceGroup \
    --plan myAppServicePlan \
    --name my-fastapi-app \
    --deployment-container-image-name myacr.azurecr.io/fastapi-demo:latest
```

23.2 Option 2: Azure Functions (Serverless)

Azure Functions is Microsoft's serverless platform — equivalent to AWS Lambda.

Project Structure:

```
my-function-app/
├── function_app.py          # Main entry point
├── WrapperFunction/
|   └── __init__.py          # FastAPI app code
├── requirements.txt
├── host.json
└── local.settings.json
```

function_app.py:

python

```python
import azure.functions as func
from WrapperFunction import app as fastapi_app

app = func.AsgiFunctionApp(
    app=fastapi_app,
    http_auth_level=func.AuthLevel.ANONYMOUS
)
```

WrapperFunction/init.py:

python

```python
import fastapi

app = fastapi.FastAPI()

@app.get("/sample")
async def index():
    return {"info": "Try /hello/YourName for parameterized route."}

@app.get("/hello/{name}")
async def get_name(name: str):
    return {"name": name}
```

requirements.txt:

```
azure-functions
fastapi
```

Deploy:

```bash
# Install Azure Functions Core Tools
# Test locally
func host start

# Deploy to Azure
az functionapp create \
    --resource-group myResourceGroup \
    --consumption-plan-location eastus \
    --runtime python \
    --functions-version 4 \
    --name my-fastapi-function \
    --storage-account mystorage \
    --os-type Linux

func azure functionapp publish my-fastapi-function
```

Access your API:

- Base: `https://my-fastapi-function.azurewebsites.net/sample`
- Parameterized: `https://my-fastapi-function.azurewebsites.net/hello/World`

23.3 Option 3: Azure Container Apps

Azure Container Apps is the equivalent of AWS ECS Fargate — serverless containers.

```bash
# Create Container App environment
az containerapp env create \
    --name myEnvironment \
    --resource-group myResourceGroup \
    --location eastus

# Deploy container app
az containerapp create \
    --name my-fastapi-app \
    --resource-group myResourceGroup \
    --environment myEnvironment \
    --image myacr.azurecr.io/fastapi-demo:latest \
    --target-port 80 \
    --ingress external \
    --min-replicas 1 \
    --max-replicas 5
```

23.4 Azure with PostgreSQL — Full Stack Deployment

Microsoft provides a one-command deployment for FastAPI + PostgreSQL:

```bash
# Clone sample app
git clone https://github.com/Azure-Samples/msdocs-fastapi-postgresql-sample-app
cd msdocs-fastapi-postgresql-sample-app

# Deploy everything with Azure Developer CLI
azd auth login
azd init
azd up
```

This creates: App Service + PostgreSQL + GitHub Actions CI/CD automatically.

Comparison Table — Azure Deployment Options:

Feature	App Service	Azure Functions	Container Apps
Type	PaaS	Serverless	Serverless Containers
Cost	From ~$13/month	Pay per execution (free tier: 1M/month)	Pay per usage
Scaling	Manual/Auto	Automatic	Automatic
Complexity	Low	Medium	Medium
Best For	Web apps, APIs	Event-driven, lightweight APIs	Microservices

---

24. Docker and Containerization

24.1 Basic Dockerfile

dockerfile

```dockerfile
FROM python:3.12-slim

WORKDIR /app

# Copy and install dependencies first (for caching)
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copy application code
COPY . .

# Expose the port
EXPOSE 8000

# Run the application
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

24.2 Docker Compose (FastAPI + PostgreSQL + Redis)

yaml

```yaml
# docker-compose.yml
version: '3.8'

services:
  api:
    build: .
    ports:
      - "8000:8000"
    environment:
      - DATABASE_URL=postgresql://user:password@db:5432/appdb
      - REDIS_URL=redis://redis:6379
    depends_on:
      - db
      - redis

  db:
    image: postgres:15
    environment:
      - POSTGRES_USER=user
      - POSTGRES_PASSWORD=password
      - POSTGRES_DB=appdb
    volumes:
      - postgres_data:/var/lib/postgresql/data
    ports:
      - "5432:5432"

  redis:
    image: redis:7-alpine
    ports:
      - "6379:6379"

volumes:
  postgres_data:
```

bash

```
# Build and run
docker-compose up --build

# Run in background
docker-compose up -d

# Stop
docker-compose down

# View logs
docker-compose logs -f api
```

24.3 Production Dockerfile (Multi-stage Build)

dockerfile

```
# Build stage
FROM python:3.12-slim as builder

WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir --user -r requirements.txt

# Production stage
FROM python:3.12-slim

WORKDIR /app
COPY --from=builder /root/.local /root/.local
COPY . .

ENV PATH=/root/.local/bin:$PATH

EXPOSE 8000

CMD ["gunicorn", "main:app", "-k", "uvicorn.workers.UvicornWorker", \
     "--bind", "0.0.0.0:8000", "--workers", "4"]
```

---

25. Performance Optimization

25.1 Caching with Redis

python

```python
import aioredis
import json

redis = None

@app.on_event("startup")
async def startup():
    global redis
    redis = await aioredis.from_url("redis://localhost")

@app.get("/items/{item_id}")
async def get_item(item_id: int):
    # Check cache first
    cached = await redis.get(f"item:{item_id}")
    if cached:
        return json.loads(cached)

    # If not cached, get from database
    item = await fetch_from_db(item_id)

    # Store in cache for 5 minutes
    await redis.set(f"item:{item_id}", json.dumps(item), ex=300)
    return item
```

25.2 Connection Pooling

```python
from sqlalchemy import create_engine

engine = create_engine(
    DATABASE_URL,
    pool_size=20,          # Maximum connections in pool
    max_overflow=10,       # Extra connections allowed
    pool_timeout=30,       # Seconds to wait for connection
    pool_recycle=1800,     # Recycle connections after 30 minutes
)
```

25.3 Response Compression

```python
from fastapi.middleware.gzip import GZipMiddleware

app.add_middleware(GZipMiddleware, minimum_size=1000)
```

25.4 Profiling

```bash
# Install profiling tools
pip install py-spy

# Profile running application
py-spy top --pid <PID>

# Generate flame graph
py-spy record -o profile.svg --pid <PID>
```

---

26. Interview Questions - Basic Level

Q1. What is FastAPI and why is it called "Fast"?

Answer: FastAPI is a modern Python web framework for building APIs. It is called "Fast" for two reasons: first, it delivers very high performance comparable to Node.js and Go because it's built on Starlette (ASGI) and uses Pydantic for data handling. Second, it speeds up development time by 200-300% because it automatically handles validation, documentation, and serialization through Python type hints.

Q2. What are the main components that FastAPI is built on?

Answer: FastAPI is built on two main libraries: **Starlette** (which handles all the web-related parts like routing, middleware, WebSockets, and async support) and **Pydantic** (which handles data validation, serialization, and settings management using Python type hints).

Q3. How do you install and run a basic FastAPI application?

Answer: Install with `pip install "fastapi[standard]"`, create a `main.py` file with a FastAPI app instance and routes, then run with `fastapi dev main.py` or `uvicorn main:app --reload`. The `--reload` flag auto-restarts the server when code changes.

Q4. What is the difference between path parameters and query parameters?

Answer: Path parameters are part of the URL path itself (like `/users/42` where 42 is the path parameter). They are defined using curly braces in the route. Query parameters come after `?` in the URL (like `/users?age=25&active=true`). Path parameters are usually required and identify a specific resource, while query parameters are often optional and filter/modify the result.

Q5. What is Pydantic and why is it important in FastAPI?

Answer: Pydantic is a data validation library that uses Python type hints. In FastAPI, it automatically validates request data, converts data types, generates error messages when data is

wrong, and creates JSON schema for API documentation. You define data shapes using Pydantic `BaseModel` classes, and FastAPI handles the rest.

Q6. What is Uvicorn?

Answer: Uvicorn is an ASGI (Asynchronous Server Gateway Interface) server that runs FastAPI applications. It's built on `uvloop` and `httptools` for very fast performance. It's the "engine" that actually serves your FastAPI app to the web.

Q7. How does FastAPI generate automatic documentation?

Answer: FastAPI reads your route definitions, type hints, and Pydantic models to automatically create OpenAPI specifications. It then provides two interactive documentation interfaces: Swagger UI at `/docs` (where you can test endpoints) and ReDoc at `/redoc` (clean readable documentation). No extra code is needed.

Q8. What is the difference between `def` and `async def` in FastAPI?

Answer: Routes defined with `async def` can use `await` for non-blocking I/O operations (database calls, API calls). Routes defined with regular `def` are run in a separate thread pool by FastAPI so they don't block the main event loop. Use `async def` for I/O-bound work with async libraries, and `def` for CPU-bound work or when using synchronous libraries.

Q9. What are the common HTTP methods used in FastAPI?

Answer: GET (read data), POST (create data), PUT (replace data entirely), PATCH (update partially), DELETE (remove data). FastAPI provides decorators for each: `@app.get()`, `@app.post()`, `@app.put()`, `@app.patch()`, `@app.delete()`.

Q10. How do you handle request body in FastAPI?

Answer: You define a Pydantic model and use it as a parameter type in your route function. FastAPI automatically reads the JSON body, validates it against the model, and passes the validated object to your function. If validation fails, FastAPI returns a 422 error with details.

---

27. Interview Questions - Intermediate Level

Q11. Explain Dependency Injection in FastAPI with an example.

Answer: Dependency Injection in FastAPI is a system where functions receive their required objects (dependencies) from FastAPI instead of creating them. You define a dependency function and use `Depends()` to inject it. For example, a `get_db()` function that creates and yields a database session can be injected into any route using `db: Session = Depends(get_db)`. This makes code reusable, testable, and clean. Dependencies can also depend on other dependencies (sub-dependencies), and FastAPI resolves the entire chain automatically.

Q12. What is the `response_model` parameter and why is it important?

Answer: `response_model` defines the shape of data returned by an endpoint. It's important because: (1) it filters the output to include only declared fields (so passwords or internal data aren't accidentally exposed), (2) it validates the response data, (3) it generates accurate API documentation, and (4) it converts data types. You define it as a Pydantic model and pass it to the route decorator.

Q13. How does FastAPI handle data validation errors?

Answer: When incoming data doesn't match the expected Pydantic model or type hints, FastAPI automatically returns a 422 (Unprocessable Entity) response with detailed JSON error information including the field that failed, the error type, and a human-readable message. You can also customize this behavior by adding a custom exception handler for `RequestValidationError`.

Q14. What is middleware in FastAPI and when would you use it?

Answer: Middleware is code that runs before every request reaches your route handlers and after the response is generated. It's used for cross-cutting concerns like logging, authentication, adding custom headers, measuring response time, CORS handling, and rate limiting. You create middleware using `@app.middleware("http")` decorator.

Q15. How do you implement background tasks in FastAPI?

Answer: FastAPI provides `BackgroundTasks` which you inject into your route function. You add tasks using `background_tasks.add_task(function, args)`. These tasks run AFTER the response is sent, so the user gets a quick response while the server processes tasks like sending emails or writing logs in the background. For heavy tasks, you should use Celery with Redis/RabbitMQ instead.

Q16. Explain how CORS works in FastAPI.

Answer: CORS (Cross-Origin Resource Sharing) controls which frontend domains can access your API. By default, browsers block requests from different origins. In FastAPI, you add `CORSMiddleware` and configure: `allow_origins` (which domains can access), `allow_methods` (GET, POST, etc.), `allow_headers` (custom headers allowed), and `allow_credentials` (whether cookies are allowed). This is essential when your React/Vue frontend is on a different port than your API.

Q17. How do you organize a large FastAPI project using APIRouter?

Answer: You create separate route files using `APIRouter`, each handling a specific domain (users, posts, orders). Each router file defines its routes with `@router.get()`, `@router.post()`, etc. In the main app, you include these routers using `app.include_router(router)`. You can add prefixes (like `/api/v1/users`) and tags for documentation grouping. This keeps your codebase clean and maintainable.

Q18. What is the difference between ASGI and WSGI?

Answer: WSGI is synchronous — it handles one request at a time per worker. ASGI is asynchronous — it can handle many requests concurrently. FastAPI uses ASGI (via Starlette), which makes it ideal for I/O-heavy applications, WebSockets, and real-time features. Flask and Django traditionally use WSGI, though Django now has partial ASGI support.

Q19. How do you implement pagination in FastAPI?

Answer: You can implement pagination using query parameters: `skip` (offset) and `limit` (page size). Use dependency injection for reusability. In your database query, apply `.offset(skip).limit(limit)`. You can also return total count and next/previous page URLs in the response for a complete pagination system. A common pattern is creating a `Pagination` class dependency.

Q20. How do you test FastAPI applications?

Answer: FastAPI provides `TestClient` (based on httpx) for synchronous testing and supports `pytest` with `pytest-asyncio` for async tests. You create a `TestClient(app)` instance and make requests using `client.get()`, `client.post()`, etc. For testing with different dependencies (like test databases), use `app.dependency_overrides` to replace real dependencies with test versions.

---

28. Interview Questions - Advanced Level

Q21. How do you implement OAuth2 with JWT in FastAPI?

Answer: FastAPI provides `OAuth2PasswordBearer` for the OAuth2 flow. You create a `/token` endpoint that authenticates users (verify username/password using `passlib` for hashing), generates a JWT token (using `python-jose`), and returns it. For protected routes, you create a dependency that extracts the token from the `Authorization: Bearer <token>` header, decodes it, validates claims (expiration, subject), and returns the current user. If invalid, you raise an HTTPException with 401 status.

Q22. How would you implement rate limiting in FastAPI?

Answer: You can implement rate limiting using: (1) `slowapi` library which provides decorators like `@limiter.limit("5/minute")`, (2) Custom middleware that tracks request counts using Redis, or (3) External services like API Gateway (AWS/Azure). For production, Redis-based rate limiting is preferred because it works across multiple server instances. You track requests by IP address or API key and return 429 (Too Many Requests) when the limit is exceeded.

Q23. Explain WebSocket implementation in FastAPI for a chat application.

Answer: FastAPI has built-in WebSocket support. You create a WebSocket endpoint using `@app.websocket("/ws")`. You implement a `ConnectionManager` class that maintains a list of active connections. When a client connects, it's added to the list. When a message is received, it's

broadcast to all connected clients. When disconnected, the client is removed. For scaling across multiple servers, you'd use Redis pub/sub to broadcast messages between server instances.

Q24. How do you handle database migrations in a FastAPI project?

Answer: You use Alembic for database migrations. After initializing with `alembic init alembic`, you configure it to point to your database and models. When you change SQLAlchemy models, run `alembic revision --autogenerate -m "description"` to create a migration script, then `alembic upgrade head` to apply it. For rollback, use `alembic downgrade -1`. In CI/CD pipelines, run migrations automatically before deploying.

Q25. How do you implement caching in FastAPI?

Answer: Caching can be implemented at multiple levels: (1) Response caching using Redis — check cache before hitting the database, store results with TTL. (2) In-memory caching using `functools.lru_cache` for configuration and settings. (3) HTTP caching using Cache-Control headers and ETags. (4) For distributed systems, use Redis or Memcached. You can create a caching dependency that wraps your database queries. Cache invalidation strategy depends on whether data is read-heavy (long TTL) or write-heavy (short TTL or event-based invalidation).

Q26. How would you deploy FastAPI in a microservices architecture?

Answer: Each microservice is a separate FastAPI application with its own database, Docker container, and deployment. Services communicate via REST APIs (using httpx) or message queues (RabbitMQ/Kafka). You use Docker Compose for local development and Kubernetes/ECS for production. An API Gateway (like Kong or AWS API Gateway) sits in front to handle routing, authentication, and rate limiting. Each service has its own CI/CD pipeline and can be scaled independently.

Q27. How do you handle file uploads in FastAPI efficiently for large files?

Answer: For large files, use chunked uploads with `UploadFile` (which uses `SpooledTemporaryFile` that stores in memory for small files and disk for large ones). Read the file in chunks using `await file.read(chunk_size)` to avoid loading the entire file into memory. For very large files, implement multipart upload with progress tracking, use background tasks for processing, and consider direct upload to S3/Azure Blob Storage using pre-signed URLs.

Q28. How do you implement logging and monitoring in a production FastAPI app?

Answer: Use Python's built-in `logging` module configured with structured JSON output. Create logging middleware to capture request/response details. For production monitoring, integrate with Prometheus (metrics collection), Grafana (dashboards), or cloud-native tools (CloudWatch, Azure Monitor). Implement health check endpoints (`/health`), readiness probes for Kubernetes, and distributed tracing with OpenTelemetry for microservices.

Q29. How do you optimize FastAPI for handling high concurrency?

Answer: Key strategies include: (1) Use async drivers for everything (asyncpg, motor, aioredis). (2) Connection pooling for database and external services. (3) Redis caching to reduce database

load. (4) Run with multiple Gunicorn workers: `gunicorn -k uvicorn.workers.UvicornWorker -w $(nproc)`. (5) Use `response_model` to limit serialized data. (6) Enable response compression with GZipMiddleware. (7) Profile with py-spy to find bottlenecks. (8) Use CDN for static assets. (9) Consider read replicas for read-heavy workloads.

Q30. Explain the lifespan events in FastAPI and their use cases.

Answer: Lifespan events (using `@asynccontextmanager` with the `lifespan` parameter) allow you to run code when the application starts and shuts down. Use startup for: initializing database connections, loading ML models, connecting to Redis/message queues, warming up caches. Use shutdown for: closing database connections, flushing logs, gracefully stopping background workers. This replaced the older `@app.on_event("startup")` and `@app.on_event("shutdown")` decorators.

---

29. Scenario-Based Interview Questions

Scenario 1: Design a REST API for an E-commerce Platform

Question: "Design the API endpoints for an e-commerce platform with users, products, orders, and payments."

Answer:

```python
```

```
# Users
POST   /api/v1/auth/register      # Register new user
POST   /api/v1/auth/login          # Login and get JWT token
GET    /api/v1/users/me            # Get current user profile
PUT    /api/v1/users/me            # Update profile

# Products
GET    /api/v1/products            # List products (with pagination, filters)
GET    /api/v1/products/{id}       # Get product detail
POST   /api/v1/products            # Create product (admin only)
PUT    /api/v1/products/{id}       # Update product (admin only)
DELETE /api/v1/products/{id}       # Delete product (admin only)

# Cart
GET    /api/v1/cart                # Get user's cart
POST   /api/v1/cart/items          # Add item to cart
PUT    /api/v1/cart/items/{id}     # Update cart item quantity
DELETE /api/v1/cart/items/{id}     # Remove item from cart

# Orders
POST   /api/v1/orders              # Create order from cart
GET    /api/v1/orders              # List user's orders
GET    /api/v1/orders/{id}         # Get order detail
PATCH  /api/v1/orders/{id}/cancel  # Cancel order

# Payments
POST   /api/v1/payments            # Process payment for order
GET    /api/v1/payments/{id}       # Get payment status
```

Key Design Decisions:

- Version the API (`/api/v1/`) for future changes
- Use JWT authentication with role-based access (admin vs user)
- Pagination using `skip` and `limit` query parameters
- Search/filter products using query parameters
- Background tasks for email notifications on order placement
- Webhook endpoint for payment provider callbacks

Scenario 2: Your API is Getting Slow Under Load

Question: "Your FastAPI application is handling 1000 requests per second and response times have increased from 50ms to 2 seconds. How would you debug and fix this?"

Answer:

Step 1 — Identify the bottleneck:

- Add timing middleware to log response times per endpoint
- Use `py-spy` to profile the running application
- Check database query times
- Monitor system resources (CPU, memory, connections)

Step 2 — Common fixes:

- If database queries are slow: Add indexes, optimize queries, implement connection pooling, add read replicas
- If external API calls are slow: Use async HTTP client (httpx), implement caching, add circuit breakers
- If it's CPU-bound: Increase worker count, offload to Celery
- Add Redis caching for frequently accessed data
- Enable response compression
- Use `response_model` to limit serialized data size

Step 3 — Scale:

- Horizontal scaling: Add more server instances behind a load balancer
- Use async everywhere: Switch to asyncpg, aioredis
- Implement CDN for static content
- Consider read/write splitting for the database

Scenario 3: Implementing Real-Time Notifications

Question: "Design a real-time notification system where users get instant updates when their order status changes."

Answer:

```python
```

```python
# Use WebSockets for real-time communication
class NotificationManager:
    def __init__(self):
        self.connections: dict[int, WebSocket] = {}

    async def connect(self, user_id: int, websocket: WebSocket):
        await websocket.accept()
        self.connections[user_id] = websocket

    async def notify_user(self, user_id: int, message: dict):
        if user_id in self.connections:
            await self.connections[user_id].send_json(message)

manager = NotificationManager()

@app.websocket("/ws/notifications/{user_id}")
async def notification_ws(websocket: WebSocket, user_id: int):
    await manager.connect(user_id, websocket)
    try:
        while True:
            await websocket.receive_text()  # Keep connection alive
    except WebSocketDisconnect:
        del manager.connections[user_id]

# When order status changes
@app.patch("/orders/{order_id}/status")
async def update_order_status(order_id: int, status: str):
    order = update_order(order_id, status)
    # Send real-time notification
    await manager.notify_user(order.user_id, {
        "type": "order_update",
        "order_id": order_id,
        "status": status
    })
    return order
```

For multi-server deployment, use Redis pub/sub so notifications reach users connected to any server.

Scenario 4: Securing an API with Multiple Auth Methods

Question: "Your API needs to support both JWT tokens (for web/mobile) and API keys (for third-party integrations). How would you implement this?"

Answer:

```python
```

```python
from fastapi import Security, HTTPException
from fastapi.security import OAuth2PasswordBearer, APIKeyHeader

oauth2_scheme = OAuth2PasswordBearer(tokenUrl="token", auto_error=False)
api_key_header = APIKeyHeader(name="X-API-Key", auto_error=False)

async def get_current_user(
    token: str = Security(oauth2_scheme),
    api_key: str = Security(api_key_header)
):
    # Try JWT first
    if token:
        user = decode_jwt_token(token)
        if user:
            return user

    # Try API key
    if api_key:
        user = get_user_by_api_key(api_key)
        if user:
            return user

    raise HTTPException(status_code=401, detail="Not authenticated")
```

Scenario 5: Handling Database Connection Failures Gracefully

Question: "How would you make your FastAPI application resilient to database outages?"

Answer:

- Implement retry logic with exponential backoff using `tenacity` library
- Add health check endpoints that verify database connectivity
- Use connection pooling with proper timeout settings
- Implement circuit breaker pattern — stop trying to connect after repeated failures
- Return graceful error responses (503 Service Unavailable) instead of crashing
- Cache frequently accessed data in Redis as a fallback
- Set up monitoring alerts for database connection errors

---

30. Coding Challenges for Interviews

Challenge 1: Build a URL Shortener

```python
```

```python
import hashlib
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel, HttpUrl

app = FastAPI()
url_store = {}

class URLInput(BaseModel):
    url: HttpUrl

class URLOutput(BaseModel):
    short_url: str
    original_url: str

@app.post("/shorten", response_model=URLOutput)
def shorten_url(url_input: URLInput):
    url_str = str(url_input.url)
    short_hash = hashlib.md5(url_str.encode()).hexdigest()[:6]
    url_store[short_hash] = url_str
    return URLOutput(
        short_url=f"http://short.url/{short_hash}",
        original_url=url_str
    )

@app.get("/{short_code}")
def redirect_url(short_code: str):
    if short_code not in url_store:
        raise HTTPException(status_code=404, detail="URL not found")
    from fastapi.responses import RedirectResponse
    return RedirectResponse(url=url_store[short_code])
```

Challenge 2: Build a Todo API with Full CRUD

python

```python
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
from typing import Optional, List
from datetime import datetime

app = FastAPI()

class TodoCreate(BaseModel):
    title: str
    description: Optional[str] = None
    priority: int = 1  # 1=low, 2=medium, 3=high

class TodoUpdate(BaseModel):
    title: Optional[str] = None
    description: Optional[str] = None
    completed: Optional[bool] = None
    priority: Optional[int] = None

class Todo(BaseModel):
    id: int
    title: str
    description: Optional[str]
    completed: bool
    priority: int
    created_at: str

todos_db = {}
counter = 0

@app.post("/todos", response_model=Todo, status_code=201)
def create_todo(todo: TodoCreate):
    global counter
    counter += 1
    new_todo = Todo(
        id=counter,
        title=todo.title,
        description=todo.description,
        completed=False,
        priority=todo.priority,
        created_at=datetime.now().isoformat()
    )
    todos_db[counter] = new_todo
    return new_todo

@app.get("/todos", response_model=List[Todo])
def get_todos(completed: Optional[bool] = None, priority: Optional[int] = None):
```

```python
    result = list(todos_db.values())
    if completed is not None:
        result = [t for t in result if t.completed == completed]
    if priority is not None:
        result = [t for t in result if t.priority == priority]
    return result

@app.get("/todos/{todo_id}", response_model=Todo)
def get_todo(todo_id: int):
    if todo_id not in todos_db:
        raise HTTPException(status_code=404, detail="Todo not found")
    return todos_db[todo_id]

@app.patch("/todos/{todo_id}", response_model=Todo)
def update_todo(todo_id: int, updates: TodoUpdate):
    if todo_id not in todos_db:
        raise HTTPException(status_code=404, detail="Todo not found")
    todo = todos_db[todo_id]
    update_data = updates.dict(exclude_unset=True)
    for field, value in update_data.items():
        setattr(todo, field, value)
    return todo

@app.delete("/todos/{todo_id}", status_code=204)
def delete_todo(todo_id: int):
    if todo_id not in todos_db:
        raise HTTPException(status_code=404, detail="Todo not found")
    del todos_db[todo_id]
```

Challenge 3: Build a Rate-Limited API with Custom Middleware

python

```python
import time
from collections import defaultdict
from fastapi import FastAPI, Request, HTTPException

app = FastAPI()

# Simple in-memory rate limiter
request_counts = defaultdict(list)
RATE_LIMIT = 10  # requests
TIME_WINDOW = 60  # seconds

@app.middleware("http")
async def rate_limit_middleware(request: Request, call_next):
    client_ip = request.client.host
    now = time.time()

    # Clean old entries
    request_counts[client_ip] = [
        t for t in request_counts[client_ip]
        if now - t < TIME_WINDOW
    ]

    if len(request_counts[client_ip]) >= RATE_LIMIT:
        from fastapi.responses import JSONResponse
        return JSONResponse(
            status_code=429,
            content={"detail": "Too many requests. Please try again later."},
            headers={"Retry-After": str(TIME_WINDOW)}
        )

    request_counts[client_ip].append(now)
    response = await call_next(request)
    response.headers["X-RateLimit-Remaining"] = str(
        RATE_LIMIT - len(request_counts[client_ip])
    )
    return response
```

Challenge 4: Implement a Simple Cache Decorator

python

```python
import time
import hashlib
import json
from functools import wraps

cache_store = {}

def cached(ttl_seconds: int = 300):
    def decorator(func):
        @wraps(func)
        async def wrapper(*args, **kwargs):
            # Create cache key from function name and arguments
            key_data = f"{func.__name__}:{args}:{kwargs}"
            cache_key = hashlib.md5(key_data.encode()).hexdigest()

            # Check cache
            if cache_key in cache_store:
                data, timestamp = cache_store[cache_key]
                if time.time() - timestamp < ttl_seconds:
                    return data

            # Execute function
            result = await func(*args, **kwargs)

            # Store in cache
            cache_store[cache_key] = (result, time.time())
            return result
        return wrapper
    return decorator

@app.get("/expensive-data")
@cached(ttl_seconds=60)
async def get_expensive_data():
    # Simulate expensive database query
    await asyncio.sleep(2)
    return {"data": "This was expensive to compute", "timestamp": time.time()}
```

Challenge 5: Build a Simple Authentication System from Scratch

python

```python
from fastapi import FastAPI, Depends, HTTPException, status
from fastapi.security import HTTPBearer, HTTPAuthorizationCredentials
from pydantic import BaseModel, EmailStr
import hashlib
import secrets
import time

app = FastAPI()
security = HTTPBearer()

# Fake database
users_db = {}
tokens_db = {}

class UserRegister(BaseModel):
    email: str
    password: str
    name: str

class UserLogin(BaseModel):
    email: str
    password: str

def hash_password(password: str) -> str:
    return hashlib.sha256(password.encode()).hexdigest()

def create_token(user_email: str) -> str:
    token = secrets.token_hex(32)
    tokens_db[token] = {
        "email": user_email,
        "created_at": time.time(),
        "expires_at": time.time() + 3600  # 1 hour
    }
    return token

def get_current_user(credentials: HTTPAuthorizationCredentials = Depends(security)):
    token = credentials.credentials
    if token not in tokens_db:
        raise HTTPException(status_code=401, detail="Invalid token")
    token_data = tokens_db[token]
    if time.time() > token_data["expires_at"]:
        del tokens_db[token]
        raise HTTPException(status_code=401, detail="Token expired")
    return users_db[token_data["email"]]

@app.post("/register")
```

```python
def register(user: UserRegister):
    if user.email in users_db:
        raise HTTPException(status_code=400, detail="Email already registered")
    users_db[user.email] = {
        "email": user.email,
        "name": user.name,
        "password_hash": hash_password(user.password)
    }
    return {"message": "User registered successfully"}

@app.post("/login")
def login(user: UserLogin):
    if user.email not in users_db:
        raise HTTPException(status_code=401, detail="Invalid credentials")
    stored_user = users_db[user.email]
    if stored_user["password_hash"] != hash_password(user.password):
        raise HTTPException(status_code=401, detail="Invalid credentials")
    token = create_token(user.email)
    return {"access_token": token, "token_type": "bearer"}

@app.get("/profile")
def get_profile(current_user: dict = Depends(get_current_user)):
    return {"email": current_user["email"], "name": current_user["name"]}
```

Quick Reference Cheat Sheet

python

```python
# === ESSENTIALS ===
from fastapi import FastAPI, Depends, HTTPException, status
from fastapi import Query, Path, Body, Header, Cookie, Form, File, UploadFile
from fastapi import Request, Response, BackgroundTasks, WebSocket
from fastapi.responses import JSONResponse, HTMLResponse, FileResponse, RedirectResp
from fastapi.middleware.cors import CORSMiddleware
from fastapi.security import OAuth2PasswordBearer, OAuth2PasswordRequestForm, APIKey
from pydantic import BaseModel, Field, validator

# === CREATE APP ===
app = FastAPI(title="My API", version="1.0.0")

# === ROUTES ===
@app.get("/")                    # Read
@app.post("/", status_code=201) # Create
@app.put("/")                    # Full Update
@app.patch("/")                  # Partial Update
@app.delete("/", status_code=204) # Delete

# === PARAMETERS ===
# Path:  @app.get("/items/{id}") → def f(id: int)
# Query: @app.get("/items")      → def f(skip: int = 0)
# Body:  @app.post("/items")     → def f(item: Item)

# === DEPENDENCY INJECTION ===
def my_dep(): return "value"
@app.get("/", dependencies=[Depends(my_dep)])

# === BACKGROUND TASKS ===
@app.post("/")
def f(bg: BackgroundTasks):
    bg.add_task(my_function, arg1, arg2)

# === RUN ===
# Dev:  fastapi dev main.py
# Prod: gunicorn main:app -k uvicorn.workers.UvicornWorker -w 4
```

Additional Resources for Interview Preparation

1. **Official FastAPI Documentation:** https://fastapi.tiangolo.com
2. **Pydantic Documentation:** https://docs.pydantic.dev
3. **Starlette Documentation:** https://www.starlette.io

4. **SQLModel Documentation:** https://sqlmodel.tiangolo.com
5. **OpenAPI Specification:** https://swagger.io/specification/

Tips for FastAPI Interviews:

- Be ready to write code on a whiteboard or computer
- Understand when and why to use `async/await`
- Know the ecosystem (SQLAlchemy, Alembic, Pydantic, pytest)
- Prepare real-world examples from your projects
- Be ready to compare FastAPI with Flask and Django
- Understand common security concerns and how FastAPI handles them
- Know Docker basics and at least one cloud deployment option

---

Document generated on March 26, 2026
Covers FastAPI framework from basics to advanced production deployment